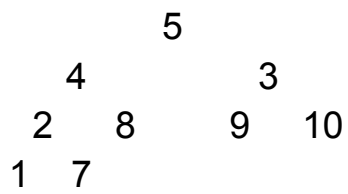
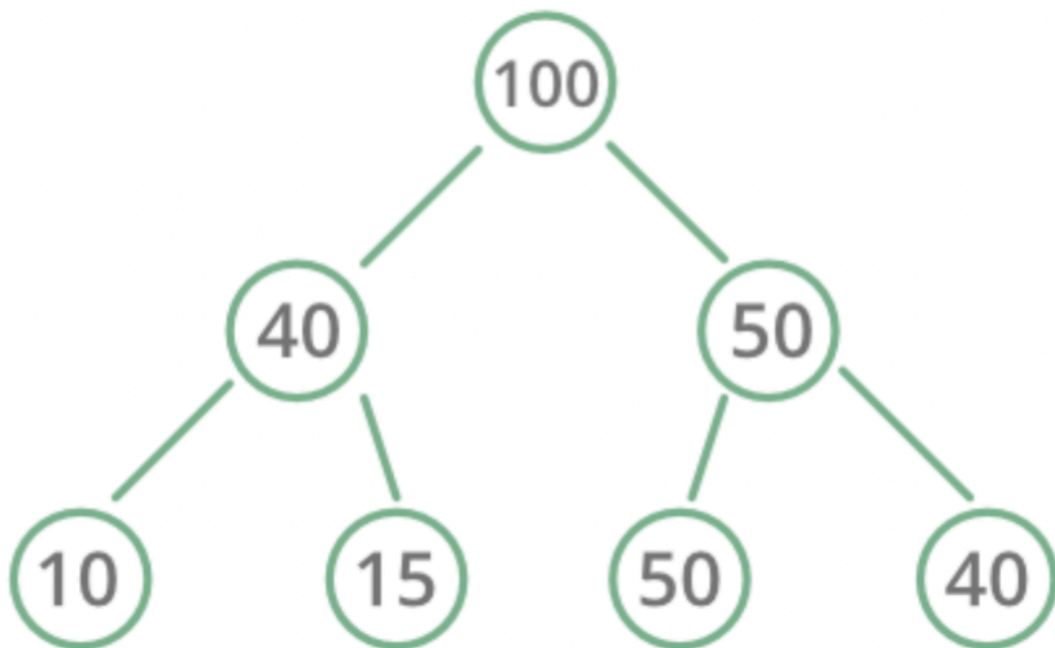
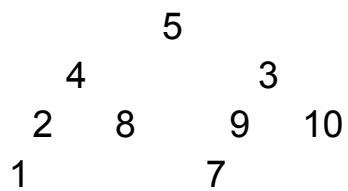


Complete Binary Tree



Non-Complete Binary Tree



0 1 2 3 4 5 6
Array: [100, 40, **50**, 10, 15, 50, 40]

For a node at index i:

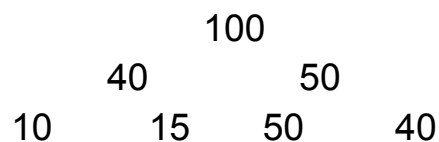
- Left child: $2*i + 1$
- Right child: $2*i + 2$
- Parent: $(i - 1) / 2$

$i = 2$

Left child: $2*2 + 1 = 5$

Right child: $2*2 + 2 = 6$

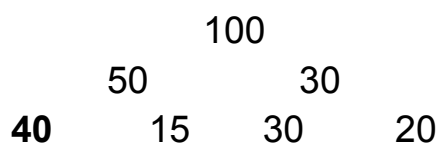
Parent: $(2 - 1)/2 = 0$



Any complete Binary Tree can be represented using an array - the level order traversal. And since Binary Heaps are complete Binary Trees, we can easily represent them using an array.

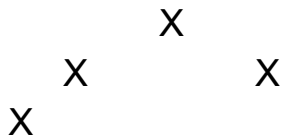
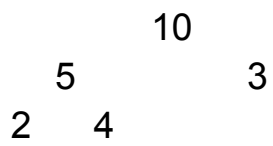
Benefits of array-based representation over the node-based representation:

1. Storage benefit - utilises much lesser storage compared to node representation.
2. No longer have to deal with references. Array traversals are much easier compared to traversals via references.
3. Access to the parent of any node is easier.



Deletion:

Input:



Step-1: Replace root with the last node and delete the last node.



Step-2: Top-Down Heapify on the root node.



Another example:

```

      50      30
    40    15  30    20
  
```

Step-1: Replace root with the last node and delete the last node.

```

      20
    50      30
  40    15  30
  
```

Step-2: Top-Down Heapify on the root node.

```

    20
    50      30
  40    15  30
  
```

```

      50
    20      30
  40    15  30
  
```

```

      50
    40      30
  20    15  30
  
```

[50, 40, 30, 20, 15, 30]

```

int maxValueIndex = index;
if (leftChildIndex < heap.size() && heap[leftChildIndex] > heap[maxValueIndex]) {
    maxValueIndex = leftChildIndex;
}
if (rightChildIndex < heap.size() && heap[rightChildIndex] > heap[maxValueIndex]) {
    maxValueIndex = rightChildIndex;
}
  
```

```

      20      30
    40    15  30

```

```

0  1  2  3  4  5
[ 50, 20, 30, 40, 15, 30 ]

```

Index = 0

leftChildIndex = 1

rightChildIndex = 2

maxValueIndex = 1

Insertion:

Input:

```

      10
     5  3
    2  4

```

Insert: 15

Output:

```

      X
     X  X
    X  X  X

```

Step-1: Insert the new node as the last node in the tree.

```

      10
     5  3

```

2 4 **15**

Step-2: Perform Bottom-Up Heapify on the newly inserted node.

 10
 5 3
 2 4 **15**

 10
 5 **15**
 2 4 3

15
 5 10
 2 4 3

Another example:

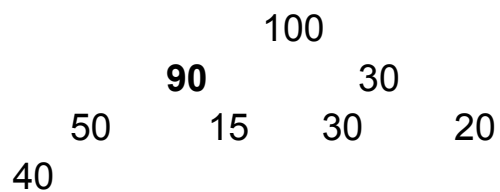
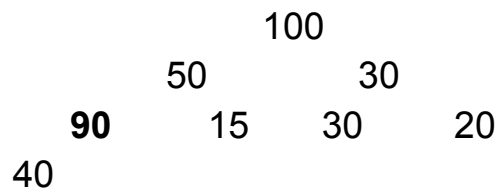
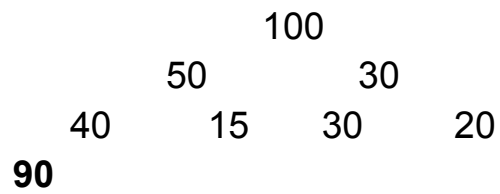
 100
 50 30
 40 15 30 20

Insert: 90

Step-1: Insert the new node as the last node in the tree.

 100
 50 30
 40 15 30 20
 90

Step-2: Perform Bottom-Up Heapify on the newly inserted node.



[100, 90, 30, 50, 15, 30, 20, 40]

TC: $O(\log(n))$

AS: $O(1)$

Summary about asymptotic complexities:

Deletion:

Time: $O(\log(n))$

AS: $O(1)$

Insertion:

Time: $O(\log(n))$

AS: $O(1)$

Kth Largest Element

[3, 5, 4, 2, 9], k = 3

Output: 4

Approach-1: Sorting based soln

[2, 3, 4, 5, 9]

TC: $O(n \cdot \log(n))$

AS: $O(1)$ if we are allowed to modify the array

(AS would be $O(n)$ if we are not allowed to modify the array)

Approach-2: Max-Heap based soln

1. Push all the elements of the array in a max-heap.
2. And then do $(K-1)$ pops on the max-heap $\rightarrow$ then k-th largest will end up as the root node.

TC: $O(n \cdot \log(n) + k \cdot \log(n)) = O(n \cdot \log(n))$

AS: $O(n)$

Approach-3: Min-Heap based soln

[3, 5, 4, 2, 9, 10], k = 3

Output: 5

K-th Largest element == **Smallest of the K largest Elements.**

(10, 9, **5**)

NOTE: We will now be using min-heap visualisations - and not the actual level-order representations.

{10, 6, 3} = means 10, 6, and 3 are there in the heap.

[**3**, 5, 4, 2, 9, 10], k = 3

MinHeap: {3}

[3, **5**, 4, 2, 9, 10], k = 3

MinHeap: {3, 5}

[3, 5, **4**, 2, 9, 10], k = 3

MinHeap: {3, 5, 4}

[3, 5, 4, **2**, 9, 10], k = 3

MinHeap: {3, 5, 4}

[3, 5, 4, 2, **9**, 10], k = 3

MinHeap: {9, 5, 4}

[3, 5, 4, 2, 9, **10**], k = 3

MinHeap: {9, 5, 10}

Final MinHeap: {9, **5**, 10}

Output: 5

Summary:

1. Insert the first K elements in the min-heap.
2. For the rest of the elements:
 - a. If $\text{arr}[i] < \text{min-so-far}$ -> skip it
 - b. If $\text{arr}[i] > \text{min-so-far}$ -> kick-out the min-so-far and insert $\text{arr}[i]$ to the list of top-k-largest-elements.

TC: $O(k \cdot \log(k) + (n-k) \cdot (\log(k))) = O((k+n-k) \cdot \log(k)) = \mathbf{O(n \cdot \log(k))}$

AS: $O(k)$

Data Structure: Priority Queue -> which internally is a heap.
For problem solving - always use this built-in data structure.